

AI-Based Restoration of Degraded Inspection Images

KLA Problem Statement PS01 · SemiCon AI Hackathon · Round 2

Team **ForgeX**

Abishek SR · Anmol BA · Hardik

VIT Vellore · abisheksundar1114@gmail.com

23.63 dB

PSNR · organisers' test set

+3.18 dB

over bicubic ×2

0.193

LPIPS (bicubic 0.466)

1.37 M parameters · 5.5 MB · 297 images in 5.9 s end-to-end · torch + numpy only

Problem Statement Addressed

Round 2 is a different problem from Round 1 — we re-derived every property rather than assuming one carried over

	Round 1	Round 2
Content	natural photographs	SEM micrographs
Training pairs	3,200	4,785
Sizes present	mixed	all 256→128; no 512×512
f90 — structure fineness	0.0945	0.2913 (3.1× finer)
Bicubic ×2 baseline	23.23 dB	20.46 dB

THE CONSTRAINT THAT SHAPED EVERYTHING

f90 is the radial frequency below which 90% of non-DC energy sits. Round-2 structure is **3.1× finer**, so 3.1× more of the signal lives above the Nyquist limit of the downsampled input — **information destroyed by the 2×2 average, not merely obscured by noise**. No architecture lifts that ceiling.
Bicubic falls 2.77 dB between rounds, so equal skill shows up as a lower score. That is why this round was spent measuring the data rather than trying models.

THE FOUR THINGS THE STATEMENT ASKS FOR

- Recover fine spatial detail lost to downsampling
- Suppress speckle noise across diverse image types
- **Generalise broadly across multiple image distributions**
- Handle the ×2 reduction in spatial resolution

The third is the whole deck. Every decision below was made against it.

IN-DISTRIBUTION vs OUT-OF-DISTRIBUTION

Every model we trained scores within 0.30 dB of every other at the noise level KLA's data actually contains. **The models only separate outside it** — and “outside it” is what the statement calls generalisation.

So we measure everything twice: once where the data lives, once where it might move to.

Idea Description

Measure the degradation, then build two generators from it — one calibrated to reality, one deliberately wider

$$\text{var}(\text{residual}) = \sigma^2_{\text{add}} + \sigma^2_{\text{mul}} \cdot m^2 + c \cdot v$$
$$\text{downsample} = \text{exact } 2 \times 2 \text{ box average}$$

m = 2x2 block mean of the clean image, v = within-block variance. **adjusted R² = 0.9987** over 288 bins / 6.55 M pixels.

SPECKLE OR SHOT NOISE? SETTLED BY BIC, NOT ASSUMED

model	form	adj R ²	BIC
M1 multiplicative	$a + b \cdot m^2 + c \cdot v$	0.9987	-1814.5
M2 Poisson / shot	$a + p \cdot m + c \cdot v$	0.9370	-696.2
M3 both terms	$a + p \cdot m + b \cdot m^2 + c \cdot v$	0.9989	-1849.5
M4 no brightness term	$a + c \cdot v$	0.1808	+37.7

ΔBIC = 1118.3 — decisive (>10 is “very strong”). M3 fits marginally better but drives the Poisson coefficient to -0.00196: negative, meaningless for a shot-noise term. It absorbs curvature, it does not count photons. So the generator multiplies, and the network *offers* $\sqrt{x}$ as one of three views rather than being built on it.

parameter	NARROW — as measured	WIDE — deliberately broader
σ_{add}	0.000 – 0.065	0.000 – 0.180
σ_{mul}	0.155 – 0.258	0.060 – 0.500
c detail term	0.026 – 0.234	0.000 – 0.450
Gamma k	14	4 – 60
optical blur	absent	σ 0.3–1.1 on 30%
downsample kernel	exact 2x2 box	box, or Gaussian on 25%

CALIBRATION IS A TEST, NOT A CLAIM

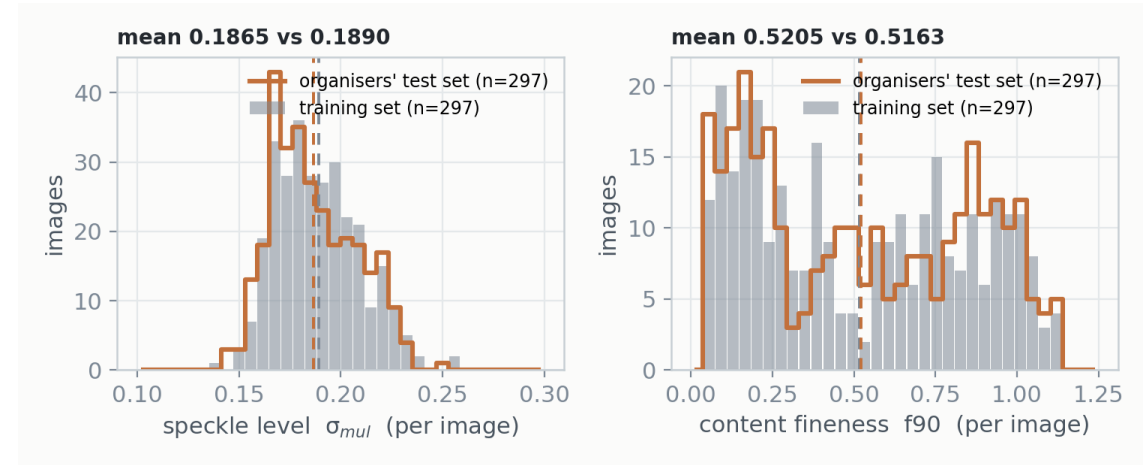
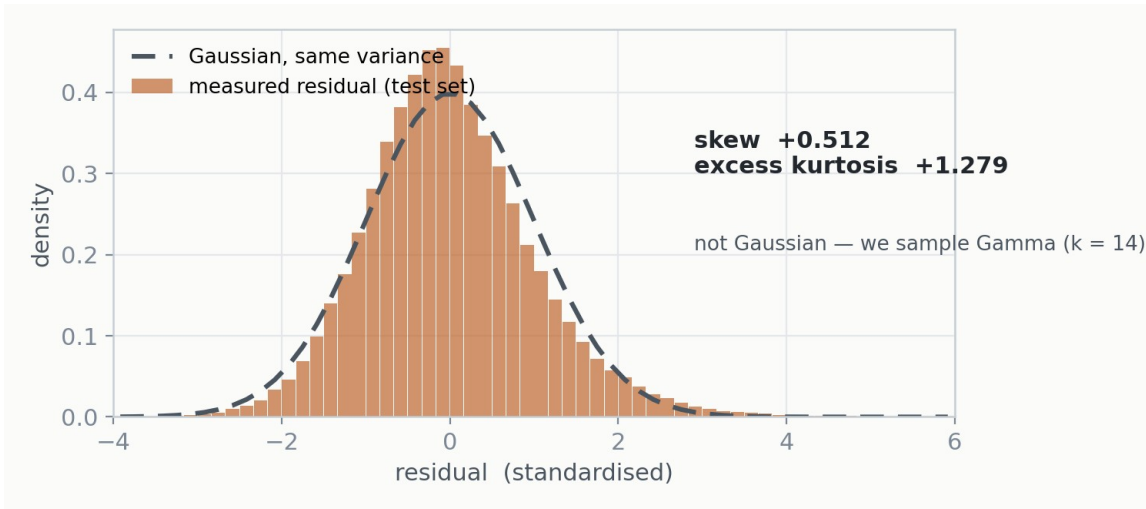
KLA real	σ_a 0.0414	σ_m 0.1863
ours NARROW	σ_a 0.0415	σ_m 0.1986
ours TRAINING	σ_a 0.0507	σ_m 0.2192

The narrow family brackets reality — that is the test. The training mixture is visibly wider on purpose.

Blur is applied to the clean image before the noise is sampled — never to the degraded output. Optics act on the image, not on the sensor. Blurring the output would correlate noise we measured to be white (lag-1 -0.052), teaching the network to remove a structure that does not exist.

Distribution Analysis

Two histograms. The first decided the generator; the second decided how we read the test set we were given.



WHY GAMMA, NOT GAUSSIAN

The residual is spatially white — lag-1 correlation -0.052 horizontal, -0.050 vertical — so independent per-pixel draws are faithful. But it is *not* Gaussian: skew **+0.51**, excess kurtosis **+1.28**.

A Gaussian generator would match the variance and miss the shape — the network would never see the heavy right tail that real speckle produces, and would learn to expect symmetric noise it will not get. We draw from a shifted Gamma with $k = 14$, which is the physically correct family for speckle and reproduces both moments.

THE SET THEY GAVE US IS IN-DISTRIBUTION

We refitted the degradation on the organisers' 297 images and compared it against 297 random training images. **Every property matches** — σ_{mul} 0.1865 vs 0.1890, f90 0.5205 vs 0.5163, skew +0.512 vs +0.539, lag-1 -0.052 vs -0.048 . Only 1–3% of test images fall outside the training p1–p99 band, which is what two samples of the *same* distribution give by construction. Zero exact overlap, verified by SHA-1.

So we did not optimise for it. The problem statement promises a test of generalisation across distributions. This set is not that test — so we treat it as a dev set and keep the robustness we would otherwise have traded away.

Synthetic Degradation Generation

Why we manufacture training pairs instead of only using the 4,785 we were handed — and why we do NOT randomise the operation order

WHY SYNTHETIC AT ALL

- 1 Unlimited pairs.** 4,785 real pairs become an unbounded stream — fresh noise every epoch, so there is nothing stable to memorise.
- 2 Control of the axis that matters.** Real pairs come at one noise level. Only a generator lets us train across a range, which is the whole out-of-distribution argument.
- 3 It is a test, not a claim.** Refitting our synthetic output with the same estimator used on the real pairs tells us whether the model of the damage is right.

HOW IT WORKS — THREE LINES, NO PIPELINE

```
m, v = block_stats(gt)
var = s_add2 + s_mul2 · m2 + c · v
out = m + sqrt(var) · Gamma(k=14)
```

m is the 2×2 block mean — the noise-free low-res reference — and v the within-block variance. We synthesise the measured variance structure **directly at low resolution**. There is no sequence of operations to order, because the statistics are generated, not simulated.

WHY WE REMOVED THE RANDOM ORDER

Our Round-1 generator randomised the order of [speckle, downsample, gaussian] — six permutations, on the reasoning that variety is safe.

Measured, that reasoning is wrong. Speckle applied BEFORE a 2×2 average has a quarter the variance of speckle applied AFTER. Those are materially different physical processes, so at most one of the six can match reality — emitting all six guarantees five are wrong, and the network spends capacity learning to invert damage that never happens.

Two more Round-1 defects the measurement caught: blur on 25% of samples (residual error rose 0.01074 → 0.01235 with it) and Gaussian noise where the real residual has skew +0.43 and excess kurtosis +0.62.

So the narrow family has no order, no blur, and draws Gamma. It is not a simulation of the microscope. It is a reproduction of the statistics we measured coming out of it.

AND THEN WE PUT THE VARIETY BACK — ON PURPOSE

The narrow family is a faithful copy of KLA's degradation. For a test set stated to contain out-of-distribution samples, faithful is exactly the wrong goal. **The evidence that forced this:** our Round-1 generator was wrong in three separate ways — yet the model trained on it fell only **1.79 dB** from σ 0.19 to 0.45, where the model trained on the CORRECT generator fell **5.10 dB**.

Being wrong in a varied way generalised better than being right in a narrow one.

So `degrade_wide()` keeps the measured functional form and adds breadth deliberately — σ_{mul} 0.060–0.500 against the observed 0.155–0.258, Gamma k 4–60 against 14, optical blur on 30%, a Gaussian downsampling kernel on 25%. Blur is applied to the clean image before noise is sampled, never to the output.

Proposed Solution

Pipeline, architecture, loss — and the measurement rule that governs every number in this deck



Architecture

Residual CNN. All work at low resolution; PixelShuffle $\times 2$ only in the final block — 4 $\times$ cheaper than upsample-first, and latency is scored.

Global bicubic skip. Predicts only the correction to a cheap baseline, so it starts at the bicubic score.

Variance-stabilising stem. Feeds the first conv raw x , $\sqrt{x}$ and $\log(1+x)$; the network picks the representation where signal-dependent noise is most uniform, instead of us assuming.

No BatchNorm. Established to hurt SR; confirmed here.

64 ch $\cdot$ 16 blocks $\cdot$ 1,368,705 params $\cdot$ 5.5 MB

Training & loss

Charbonnier + 0.15 $\cdot$ SSIM + 0.05 $\cdot$ gradient + 0.05 $\cdot$ LPIPS. No GAN.

120 epochs $\times$ 500 iters, batch 32, 64 $\times$ 64 crops, AMP, cosine schedule.

Mixture: 50% real KLA pairs / 25% wide / 25% narrow.

The gradient term sharpens edges only where the ground truth has edges — the mechanism that keeps the perceptual term honest.

--seed and --split-seed are separate on purpose. The validation split must stay fixed while training randomness varies, or a seed experiment compares two models on two different splits and measures nothing.

HOW WE DECIDE A RESULT IS REAL

0.0049 dB — two runs differing only in seed, final-5-epoch means, matched at epoch 39. We use **0.010 dB** as the threshold; anything under it is reported as null.

Never best-epoch. Best-epoch selection inflates a score by a median 0.058 dB across our six 120-epoch runs — six times the floor. Every number here is a final-5 mean.

One pass, one baseline. All models scored by the same code on the same inputs. The shared bicubic column is the checksum: if it differs between two files, those files cannot be compared.

The Loss Function

Four terms, every weight measured rather than guessed — and the guard that licenses the fourth

$$L = \text{Charbonnier}(x,y) + 0.15 \cdot (1 - \text{SSIM}) + 0.05 \cdot |\nabla x - \nabla y| + 0.05 \cdot \text{LPIPS}(x,y)$$

Charbonnier

the base term

A smooth L1. L2 averages competing hypotheses and produces the blur that L1 avoids; Charbonnier keeps L1’s robustness while staying differentiable at zero.

0.15 · SSIM

structure, not pixels

PSNR is blind to structural error that is small in magnitude. Weight tested at 0.15 and 0.30 — 0.30 (loss-ss30) scored 23.634 against 23.632 on PSNR but 0.369 LPIPS against 0.193. Kept at 0.15.

0.05 · gradient

edges where edges exist

Sharpens only where the ground truth has a gradient, so it cannot invent an edge in a flat region. This is the term that keeps the perceptual term honest. Round 1 tested 0.20: too much, rejected.

0.05 · LPIPS

the one we resisted

Perceptual losses work by synthesising plausible texture, and invented texture on an inspection image is a fabricated defect. We shipped it only after measuring that it does not fabricate.

THE LPIPS WEIGHT, SWEPT — IDENTICAL MIXTURE, ONE VARIABLE

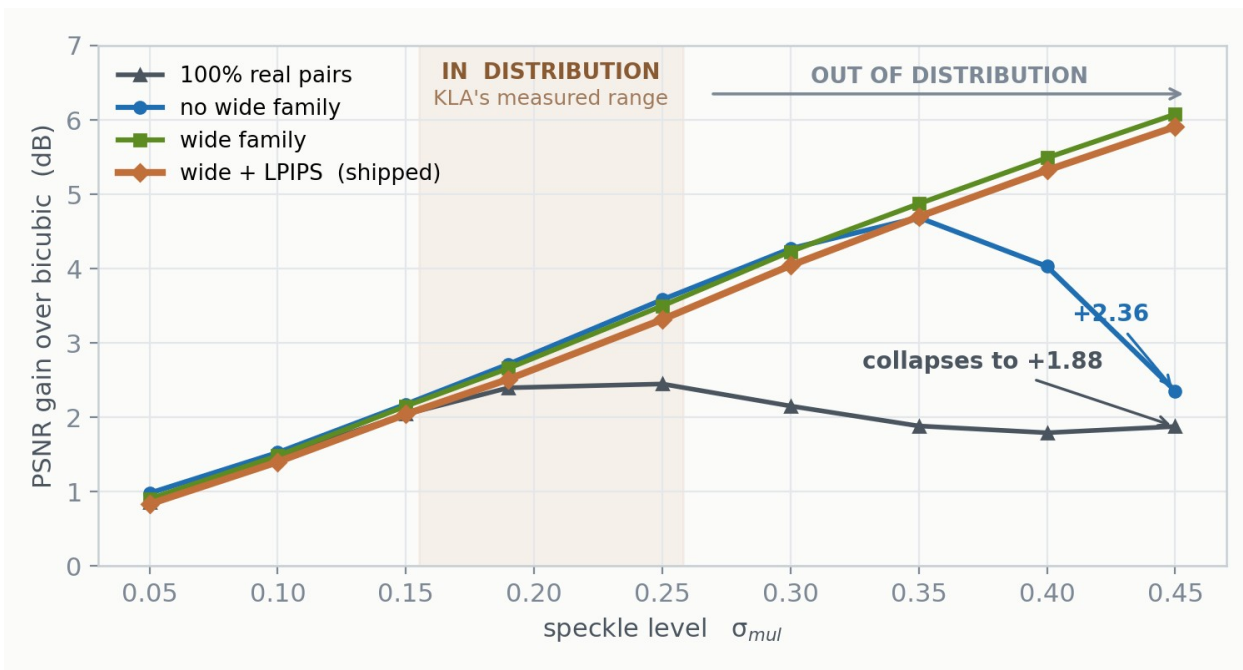
w_lpips	PSNR	SSIM	LPIPS	σ=0.45
0.00 (pr50-w50)	23.763	0.619	0.355	+6.03
0.02 (lp02)	23.685	0.607	0.220	+6.00
0.05 ★ shipped	23.632	0.608	0.193	+5.91
0.15 (loss-lp15)	23.479	0.595	0.187	+5.76

0.131 dB buys 46% of the perceptual metric. Going from 0.00 to 0.05 costs 0.131 dB of PSNR and takes LPIPS from 0.355 to 0.193. Going further, from 0.05 to 0.15, costs another 0.153 dB and buys only 0.006 — 25× worse value, and it loses 0.15 dB of noise headroom too. 0.05 is where the curve turns.

And it does not hallucinate. Energy above the input’s Nyquist — the band where content is necessarily invented — reads 0.421 for us against bicubic’s 0.375, and zero of our 297 outputs cross ratio 1.0 against bicubic’s fourteen.

Innovation & Uniqueness

Four choices that are ours — beginning with the result that reversed our own conclusion



Nine noise levels, same held-out images, identical inputs for every model. docs/noise_sweep_*.csv

For a week we believed the opposite. 100% real pairs beat every synthetic mixture on the held-out split — 23.267 vs 23.218 vs 23.134. The measurement was sound; the inference was not. $\sigma \approx 0.19$ is the one operating point where all models agree, so we were resolving 0.13 dB where there is nothing to see.

1

The wide degradation family

Three σ -steps out the real-only model is 4.03 dB behind, and it turns over at $\sigma = 0.25$ — the signature of a model that memorised a noise amplitude rather than learning the inverse problem. So we generate breadth on purpose. It costs 0.013 dB in-distribution and buys 3.72 dB outside it.

2

A measured noise floor, before any claim

Two seeds, 0.0049 dB apart. Ten of our eleven interventions failed to clear it — including 9 $\times$ more data and a 2.7 $\times$ bigger model. Publishing them as nulls is why the two that did clear it are believable.

3

A hallucination guard, not abstinence

We measure energy above the input's Nyquist — the band where content is necessarily invented. Ours 0.421 against bicubic's 0.375, and zero of our 297 outputs cross ratio 1.0 against bicubic's fourteen. That is what licenses a perceptual loss on inspection data.

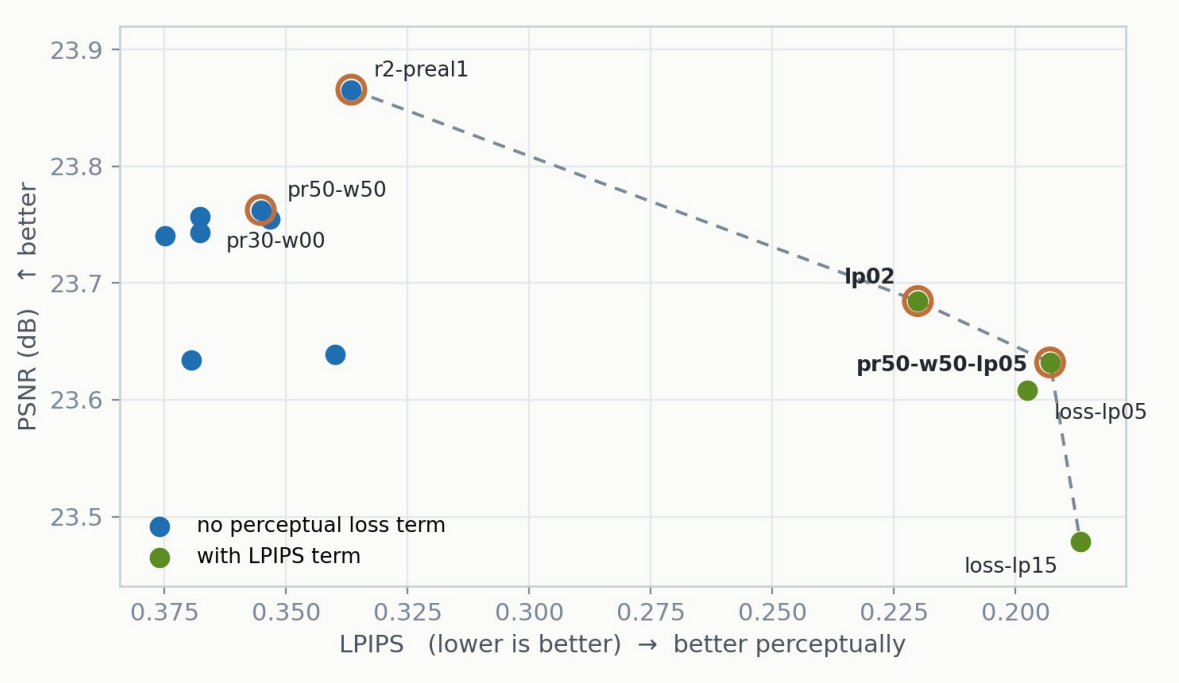
4

We used the labels the source dataset already had

We tried to recover KLA's category boundaries from spectral content and failed — one number cannot separate ten morphologies. NFFA-EUROPE, the corpus KLA's images come from, ships the labels. Ten named classes, all positive.

Model Selection

Eleven 120-epoch checkpoints from the selection queue, plus the 100%-real baseline — one scoring pass, on the organisers’ own 297 images



Circled: the efficient frontier — nothing beats them on both axes. Bicubic 20.455 / 0.5099 / 0.4655, identical for all eleven.

We do not have the best PSNR, and that is the decision. We are ninth of eleven on it. Across the whole field the worst PSNR gain still keeps 91.4% of the best; the worst LPIPS reduction keeps only 32.5%. Ship perceptual and LPIPS turns out unscored → we lose 6.9% of achievable PSNR gain. Ship non-perceptual and it IS scored → we lose 52.7% of achievable LPIPS, plus 4.03 dB of noise headroom. A 7.6× asymmetry, so we insured against the large loss.

checkpoint	PSNR	SSIM	LPIPS	$\sigma=0.45$
r2-preal1 100% real pairs	23.866	0.632	0.337	+1.88
pr50-w50 wide, no LPIPS	23.763	0.619	0.355	+6.03
pr30-w00 no wide family	23.757	0.613	0.368	+2.36
pr50-w00	23.755	0.620	0.353	—
pr30-w50	23.743	0.612	0.368	+6.08
pr30-w100 all-wide	23.740	0.611	0.375	—
lp02	23.685	0.607	0.220	+6.00
pr30-w50-hard content-balanced	23.639	0.611	0.340	—
loss-ss30 SSIM 0.30	23.634	0.616	0.369	—
pr50-w50-lp05 ★ shipped	23.632	0.608	0.193	+5.91
loss-lp05	23.608	0.604	0.198	+5.91
loss-lp15	23.479	0.595	0.187	+5.76

Results

The organisers’ own 297-image test set — zero overlap with training, verified by SHA-1

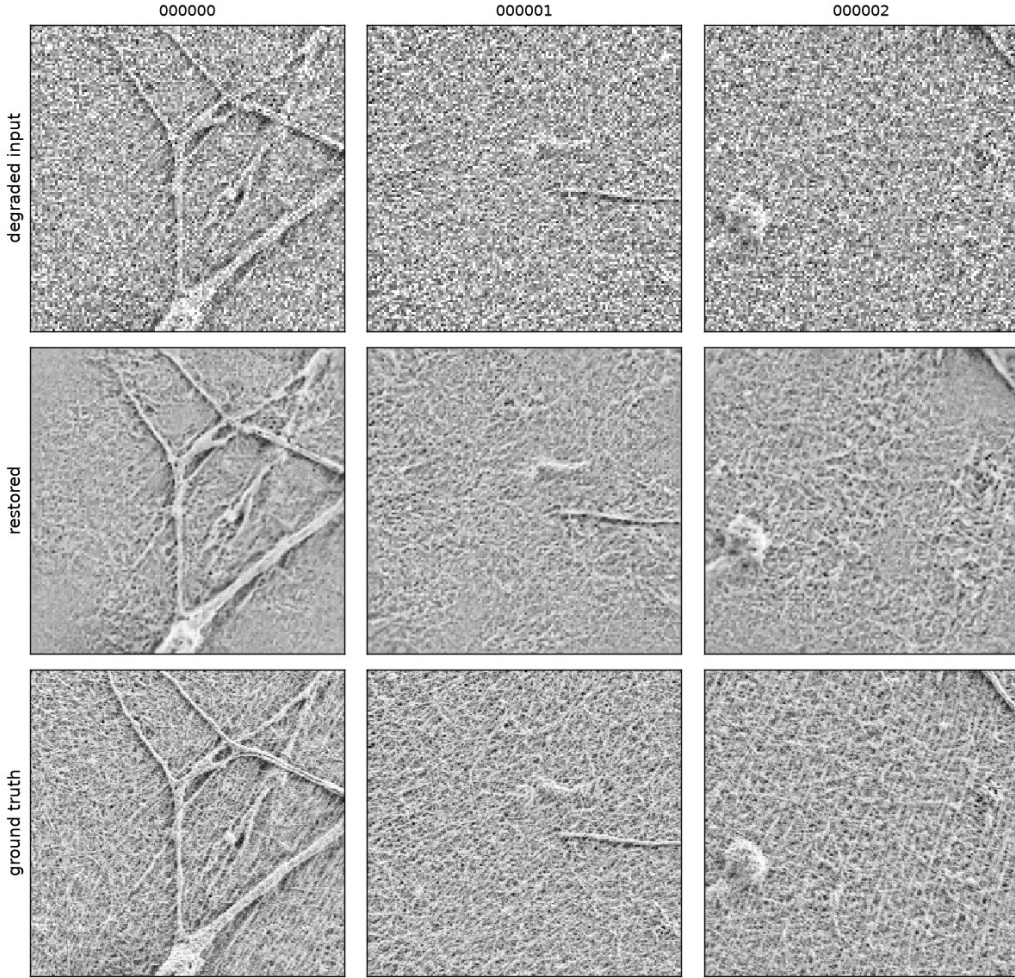
Method	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$
Bicubic $\times 2$ (no denoising)	20.455	0.5099	0.4655
Ours — 1.37 M params ★ submitted	23.632	0.6079	0.1929
gain	+3.18 dB	+0.098	−59%

+3.18 dB over bicubic

THE SAME MODEL, EVERYWHERE ELSE WE MEASURED IT

test set	n	distribution	PSNR	vs bicubic
organisers’ test set	297	in-distribution	23.632	+3.18
our held-out split	200	in-distribution	23.104	+2.88
withheld content block	478	unseen morphology	24.302	+3.65
ten NFFA morphologies	240	different corpus	—	+4.01
512×512	778	unseen scale	25.392	+4.52

The gain is largest where the model has seen least. All ten morphologies positive, mean +4.01 dB, worst +2.26 (Biological — soft organic structure has the least regular high-frequency content for a prior to exploit). The 3.01 dB spread across classes orders them exactly as the f90 analysis predicts.



degraded input → restored → ground truth

Technology & Feasibility

Everything below was measured, not estimated

	what we used	measured
Framework	PyTorch 2.13 + CUDA 12.6	torch + numpy only at inference
Training hardware	RTX 4050 Laptop, 6 GB	4.3 h · peak VRAM 0.95 GB
Selection queue	RTX 4090 pod	11 configs × 120 ep, 28 min each
Inference	297 images in 5.92 s	19.9 ms/image incl. startup
Model	1,368,705 params · 5.5 MB	no BatchNorm, no GAN

WHERE THE WALL-CLOCK ACTUALLY GOES

```
import torch 1.62 s 35%      CUDA init 1.67 s 37%      compute +  
I/O 1.17 s 26%
```

~72% is fixed process startup, not the network. Shrinking the model attacks at most the last quarter of the budget, and startup amortises over test-set size. `run.py` already overlaps what it can: CUDA init on a background thread, threaded reads, memory-mapped weights, a writer pool.

Three optimisations written, benchmarked and rejected. 8× TTA (+0.018 dB for 7.1× the time) · pinned buffers + CUDA streams (+0.04 s on a 1.94 s spread) · a `channels_last` layout fix (−0.06 s on a 0.73 s spread). All three bit-identical in output. ONNX was rejected without benchmarking: it adds a dependency the submission forbids, to save 1–2 s of startup.

ELEVEN INTERVENTIONS, MEASURED AGAINST A 0.005 dB FLOOR

change tested	effect on PSNR	verdict
random seed alone	0.0049 dB	<i>the floor itself</i>
1.8× receptive field, params matched	+0.0077 dB	null
degradation recalibration alone	−0.0162 dB	null
content-balanced sampling ($p \propto f_{90}$)	−0.0903 dB	null
8× test-time augmentation	+0.0181 dB, 7.1× slower	rejected
pinned buffers + CUDA streams	+0.04 s / 1.94 s spread	rejected
<code>channels_last</code> layout fix	−0.06 s / 0.73 s spread	rejected
4× parameters (343 k → 1.37 M)	+0.108 dB	real
rebuilt vs Round-1 generator	+0.0836 dB	real
real pairs vs synthetic, in-distribution	+0.0494 dB	real
wide degradation family, at $\sigma = 0.45$	+3.72 dB	decided it

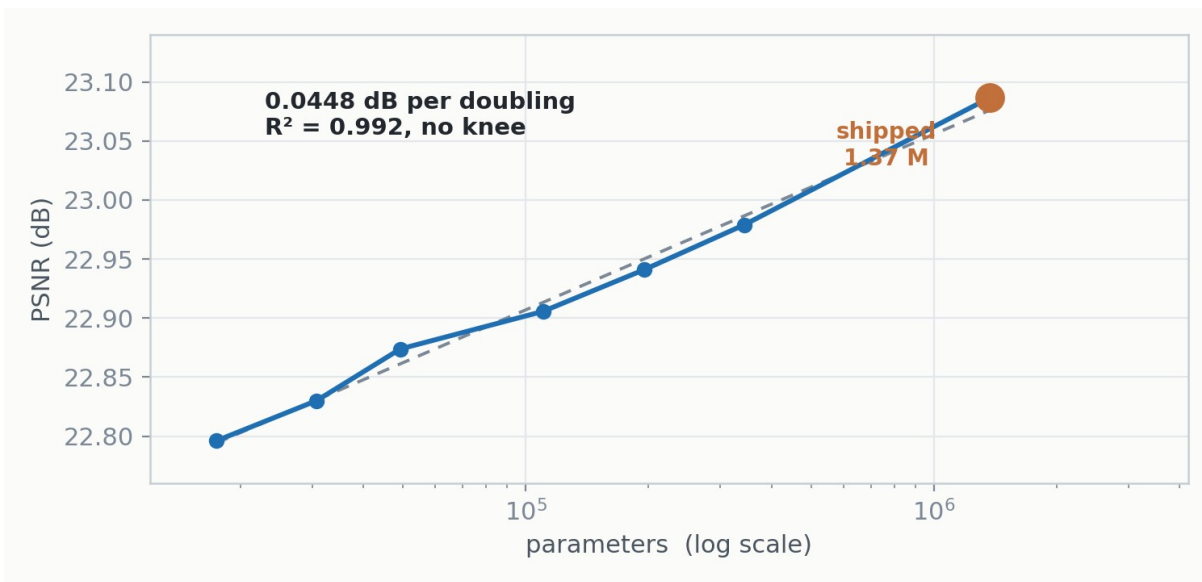
Matched *final-5* windows, or one scoring pass over identical inputs. `docs/ENGINEERING_LOG.md §10`.

Capacity: 78.4× of parameters buys 0.291 dB. Next slide.

Reproducible across machines. The shipped weights score 23.632 / 0.60791 / 0.19288 on the 4090 and 23.632 / 0.60791 / 0.19288 on the 4050 — identical to five decimals. That is what makes a queue split across two machines valid.

How Big Should the Model Be?

We trained seven sizes across a 78.4× span rather than picking one and defending it



config	parameters	PSNR	ms / image
ch12 nb4	17,449	22.7959	0.53
ch16 nb4	30,753	22.8300	0.49
ch16 nb8	49,313	22.8738	0.73
ch24 nb8	110,257	22.9056	1.27
ch32 nb8	195,393	22.9410	1.67
ch32 nb16	343,361	22.9790	2.85
ch64 nb16 ★ shipped	1,368,705	23.0865	6.50

40 epochs each, identical split, latency measured not estimated. docs/capacity_sweep.csv

THREE THINGS THIS SETTLES

- 1 “Make it bigger” is closed. 78.4× of parameters buys 0.291 dB total. Separately, 2.7× parameters bought +0.026 dB — inside the noise floor. This curve says why.
- 2 1.37 M is a decision, not a default. A 49 k model gives up 0.21 dB, is 28× smaller and 9× faster. If deployment ever demands it, that is the trade — and we know its price.
- 3 Parameter count stops predicting latency below ~30 k. ch16 nb4 is FASTER than ch12 nb4 despite 1.8× the parameters, because kernel-launch overhead dominates there.

0.045 dB per doubling. $R^2 = 0.992$. No knee anywhere. A model starved of capacity climbs steeply and then flattens — the curve has a knee, and the knee tells you where “big enough” is. This one is a straight line to three decimal places across 78.4×, which is the diagnostic that **we are not capacity-limited at all**. The bottleneck is information the 2×2 average destroyed, and no number of parameters recovers destroyed information.

Repository & Submission

Four required components, each verified by running it

#	component	file	verified by
1	Evaluation script	run.py	runs from a bare four-item directory
2	Training script	train_submitted.py	reproduces the shipped config, no flags
3	Denoised test outputs	outputs/ (297 .npy)	dtype, range and shape on every file
4	Environment spec	requirements.txt	complete pip freeze, versions pinned

tools/package_check.py runs all of it and printed PASS with max |clean - repo| = 0.000e+00 — bit-identical.

THE OUTPUT CONTRACT

Input .npy (H,W) or (H,W,1) float32, values may fall outside [0,1]. Output .npy float32 (2H,2W), same filename, guaranteed finite and clamped to [0,1]. Weights resolve relative to run.py so it works from any working directory. Nothing hardcodes a resolution — 128→256 and 256→512 both work, and mixed sizes in one directory are grouped and batched correctly. If the input path is a dataset folder rather than an images folder, run.py finds the degraded inputs one level down and says so, instead of exiting with nothing.

WHAT THE REPOSITORY CARRIES BEYOND THE FOUR FILES

docs/ENGINEERING_LOG.md — every experiment including the failures, and eight claims we published and had to withdraw · docs/queue/ — all eleven models scored on both test sets, raw CSVs · docs/tool_output/ — the analysis tools' own output · docs/superseded/ — two result files scored against a different test set, kept with a note on why they must not be quoted · run2.py, run3.py — optimisations that measured null, kept as evidence

HOW TO RUN IT

```
pip install -r requirements.txt
python run.py <in-dir> <out-dir>
```

GitHub (public)

github.com/abishek1123/forgex

Repository contains

run.py, train_submitted.py, models/model.pt, outputs/ (297 restored test images + a before/after preview), requirements.txt, the full source, and docs/ — the complete experimental record including every result we withdrew.

Round-1 archive

Full commit history from Round 1 is preserved in the same repository.

References

1 B. Lim, S. Son, H. Kim, S. Nah, K. M. Lee. “Enhanced Deep Residual Networks for Single Image Super-Resolution.” CVPRW 2017.

residual SR backbone, no BatchNorm

2 W. Shi et al. “Real-Time Single Image and Video Super-Resolution Using an Efficient Sub-Pixel Convolutional Neural Network.” CVPR 2016.

PixelShuffle upsampling

3 W.-S. Lai, J.-B. Huang, N. Ahuja, M.-H. Yang. “Deep Laplacian Pyramid Networks for Fast and Accurate Super-Resolution.” CVPR 2017.

Charbonnier loss

4 H. Zhao, O. Gallo, I. Frosio, J. Kautz. “Loss Functions for Image Restoration with Neural Networks.” IEEE Trans. Computational Imaging, 2017.

L1 + MS-SSIM loss design

5 R. Zhang, P. Isola, A. A. Efros, E. Shechtman, O. Wang. “The Unreasonable Effectiveness of Deep Features as a Perceptual Metric.” CVPR 2018.

LPIPS

6 F. J. Anscombe. “The Transformation of Poisson, Binomial and Negative-Binomial Data.” Biometrika, 1948.

variance stabilisation

7 G. Schwarz. “Estimating the Dimension of a Model.” Annals of Statistics, 1978.

BIC — used to settle the noise model

8 R. Aversa, M. H. Modarres, S. Cozzini, R. Ciancio, A. Chiusole. “The First Annotated Set of Scanning Electron Microscopy Images for Nanoscience.” Scientific Data 5, 180172, 2018.

NFFA-EUROPE — source corpus and morphology labels